Student HTTP Web API Service

DATA2410 - Assignment #3

OsloMet - Faculty of Technology, Art and Design (TKD)

Date: 04.05.2026

Harald Hammershaug (S374171)

Contents

1 Overview 2
2 Setup and Run 2
3 What we changed 4
4 Testing in Scalar 5
4.1 Seed test data (3 students) 5
4.2 Calculate grades 7
4.3 Course report 7
4.4 Delete by Id (optional) 8
4.5 Clear the database (optional) 9
5 Results 9

1 Overview

- Goal: Build two HTTP endpoints in an ASP.NET Core Web API (C#) that uses a local SQL Server database.

Endpoint 6: Calculate Grades (TO BE IMPLEMENTED)

Method	`POST`
URL	`/api/Students/calculate-grades`

Endpoint 7: Course-Wise Report (TO BE IMPLEMENTED)

Method	`GET`
URL	`/api/Students/report`

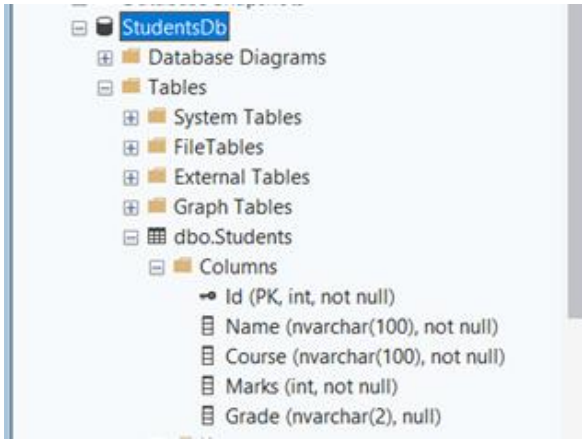
- Endpoints implemented:
    - POST /api/Students/calculate-grades – compute and save grades (A/B/C/D) for all students from Marks.
    - GET /api/Students/report – show a per-course report with total students, average marks (2 decimals), and grade counts.
- The basic CRUD endpoints (POST/GET/PUT on /api/Students) were already in the project Implemented DELETE
- CRUD mapping: POST = Create, GET = Read, PUT = Update, DELETE = Delete.

Operations

```
   GET  /api/Students
  POST  /api/Students
   GET  /api/Students/{id}
   PUT  /api/Students/{id}
DELETE  /api/Students/{id}
  POST  /api/Students/calculate-grades
   GET  /api/Students/report
```

2 Setup and Run

- Create the database in SSMS. Database name: StudentsDb



- Set the connection string in appsettings.json:
    - DefaultConnection = Server=localhost;Database=StudentsDb;Trusted_Connection=True;TrustServer Certificate=True;



- Run the API:
    - dotnet restore .\data2410-api-v1.csproj
    - dotnet run --project .\data2410-api-v1.csproj

```
PS C:\Users\Harald\Desktop\S374171_A3_StudentApi> dotnet restore .\data2410-api-v1.csproj
Restore complete (0,5s)

Build succeeded in 1,1s
PS C:\Users\Harald\Desktop\S374171_A3_StudentApi> dotnet run --project .\data2410-api-v1.csproj
Using launch settings from .\Properties\launchSettings.json...
Building...
C:\Users\Harald\Desktop\S374171_A3_StudentApi\Models\Student.cs(9,19): warning CS8618: Non-nullable
 a non-null value when exiting constructor. Consider adding the 'required' modifier or declaring th
C:\Users\Harald\Desktop\S374171_A3_StudentApi\Controllers\StudentsController.cs(79,22): warning CS8
alue.
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: https://localhost:7010
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
      Content root path: C:\Users\Harald\Desktop\S374171_A3_StudentApi
```

- Open Scalar (API docs): https://localhost:<port>/scalar/v1

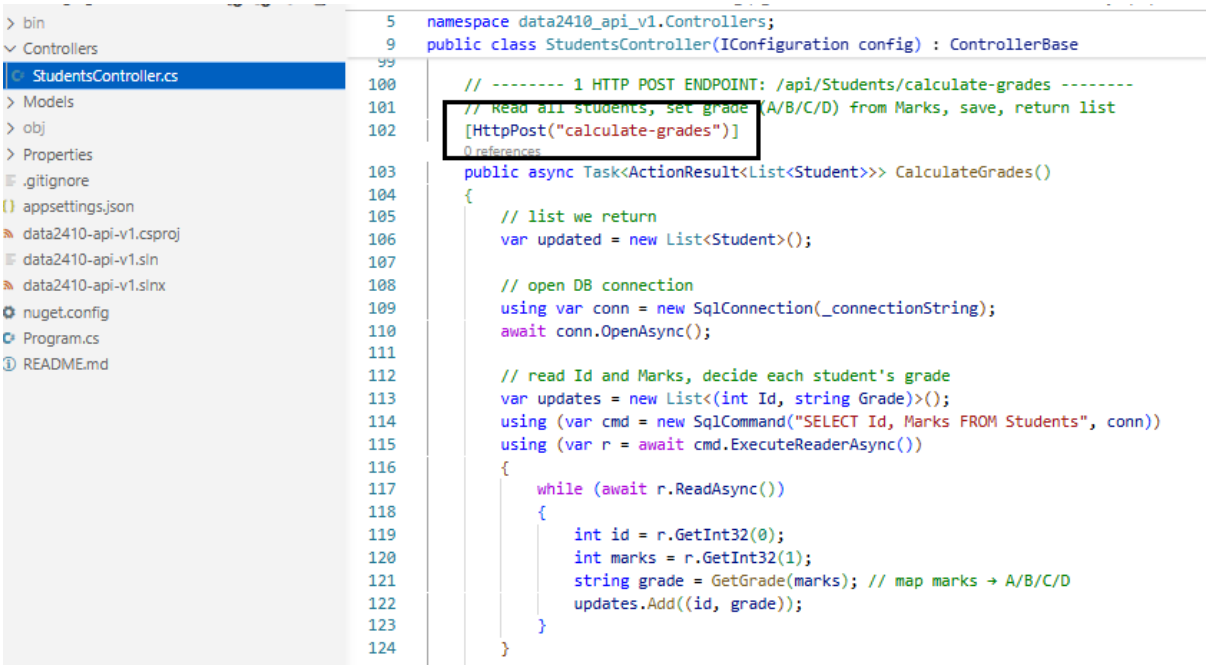


3 What we changed

File: Controllers/StudentsController.cs

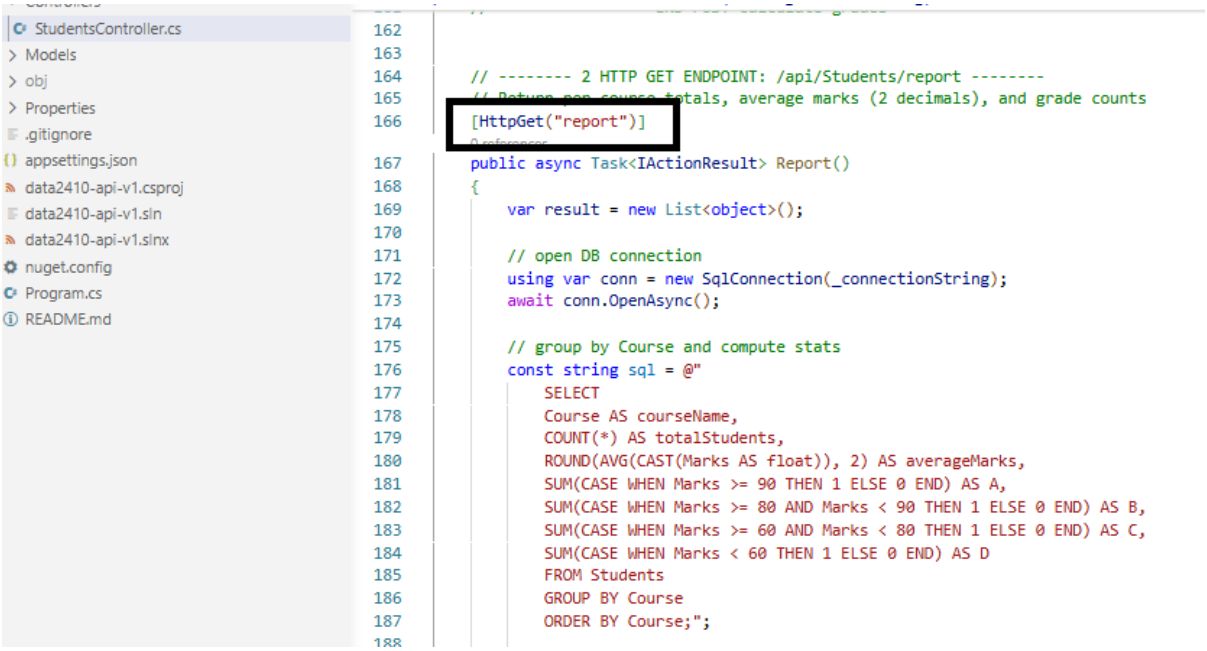
3.1 POST /api/Students/calculate-grades:

- Read all students (Id, Marks).
- Calculate Grade using GetGrade(marks):
  - A ≥ 90, B = 80–89, C = 60–79, D < 60.
- Update Grade for each student in the database.
- Return the full list with the new grades.

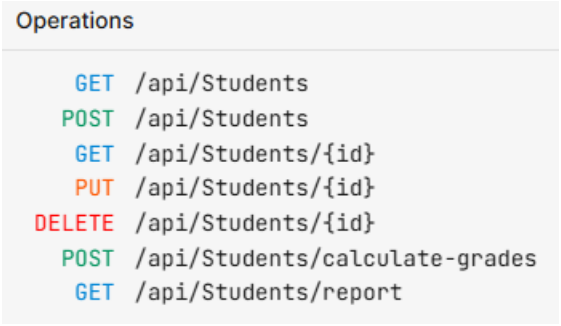


3.2 GET /api/Students/report:

- Group by Course in SQL.
- Return, for each course:
  - totalStudents (COUNT students)
  - averageMarks (ROUND to 2 decimals)
  - gradeDistribution (A, B, C, D; Sum case by mark ranges)



4 Testing in Scalar



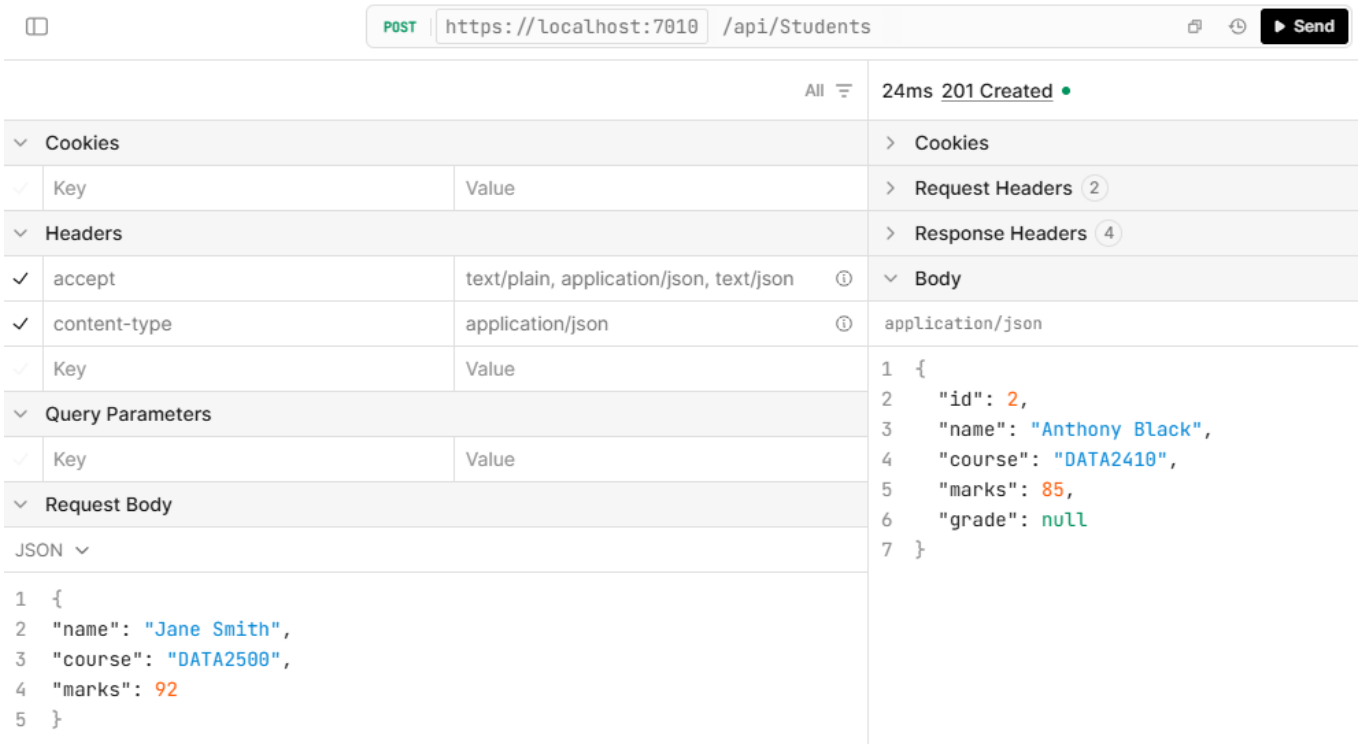
4.1 Seed test data (3 students)

- POST /api/Students (Content-Type: application/json). Send 2–3 students, for example:

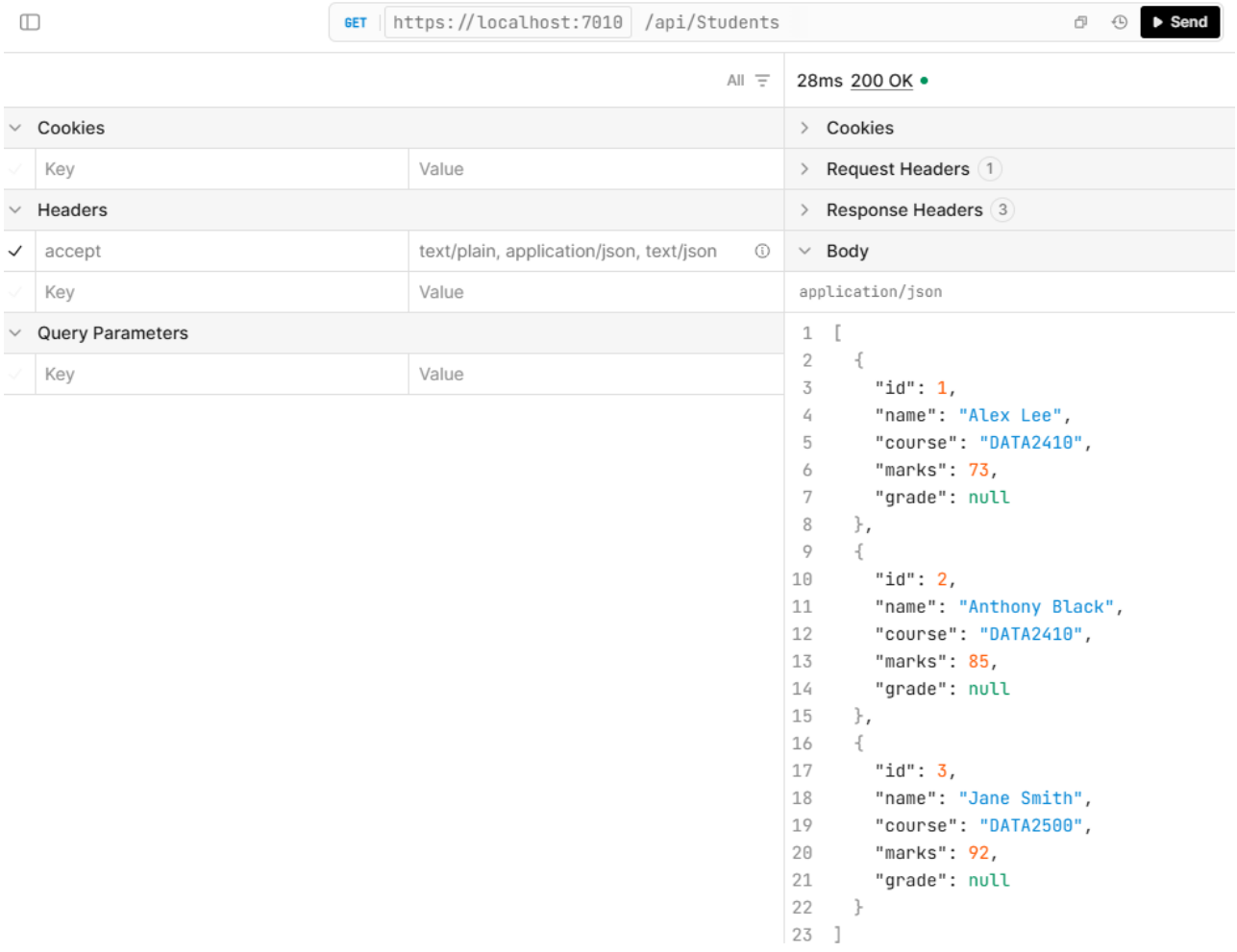
  {"name":"Alex Lee","course":"DATA2410","marks":73}

  {"name":"Anthony Black","course":"DATA2410","marks":85}

  {"name":"Jane Smith","course":"DATA2500","marks":92}

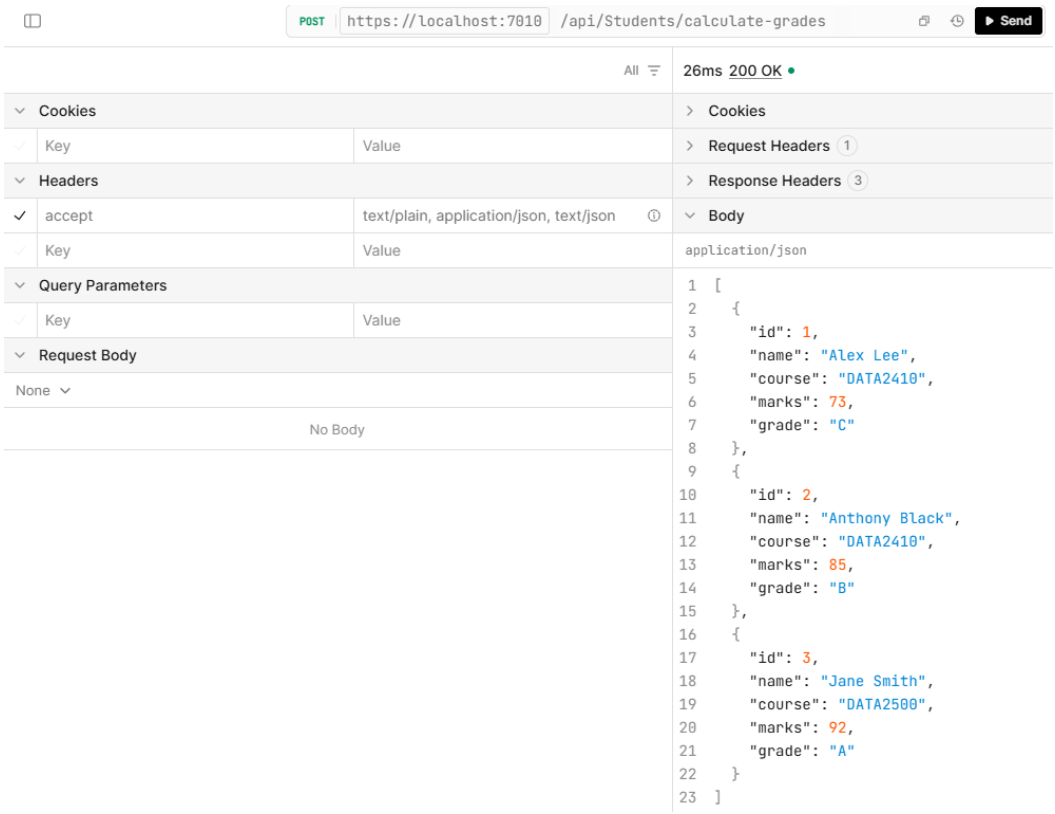


POST api/Students



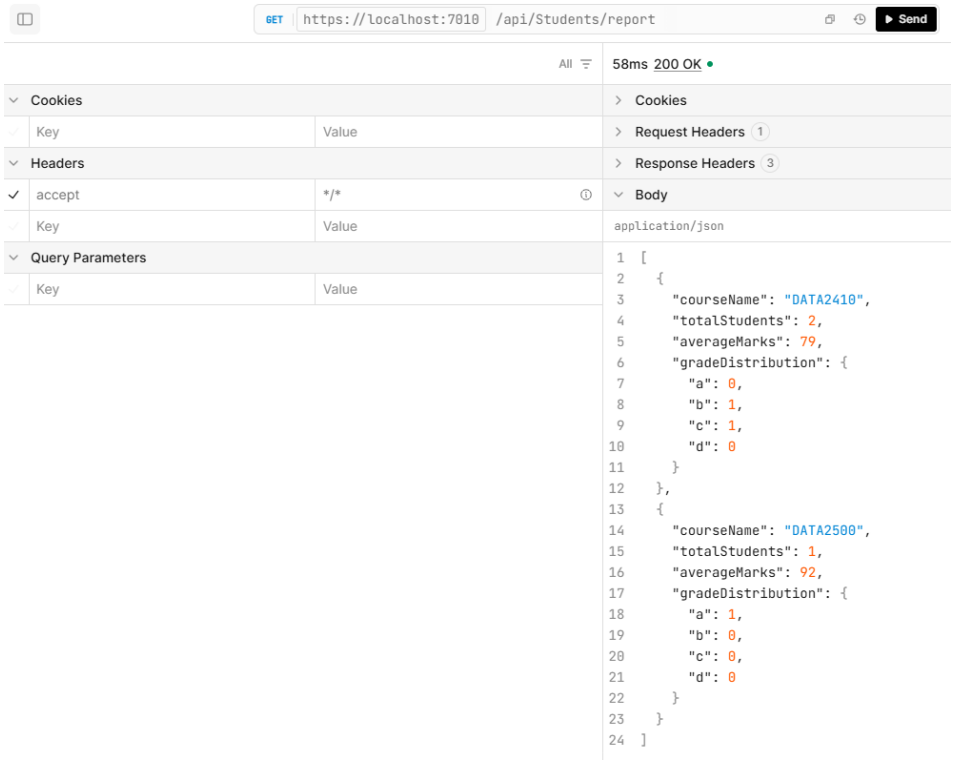
GET /api/Students

4.2 Calculate grades



POST /api/Students/calculate-grades

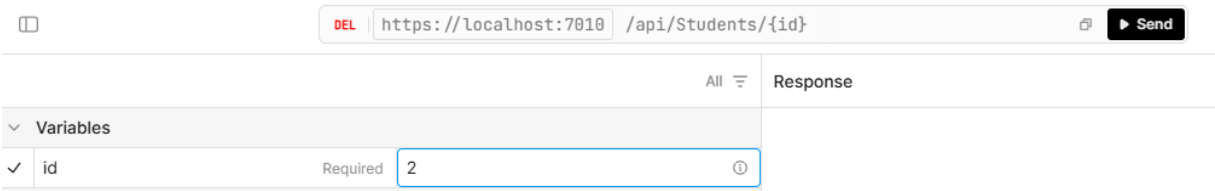
4.3 Course report



GET /api/Students/report

4.4 Delete by Id (optional)


```
[
  {
    "id": 1,
    "name": "Alex Lee",
    "course": "DATA2410",
    "marks": 73,
    "grade": "C"
  },
  {
    "id": 2,
    "name": "Anthony Black",
    "course": "DATA2410",
    "marks": 85,
    "grade": "B"
  },
  {
    "id": 3,
    "name": "Jane Smith",
    "course": "DATA2500",
    "marks": 92,
    "grade": "A"
  }
]
```

DELETE /api/Students/{Id}

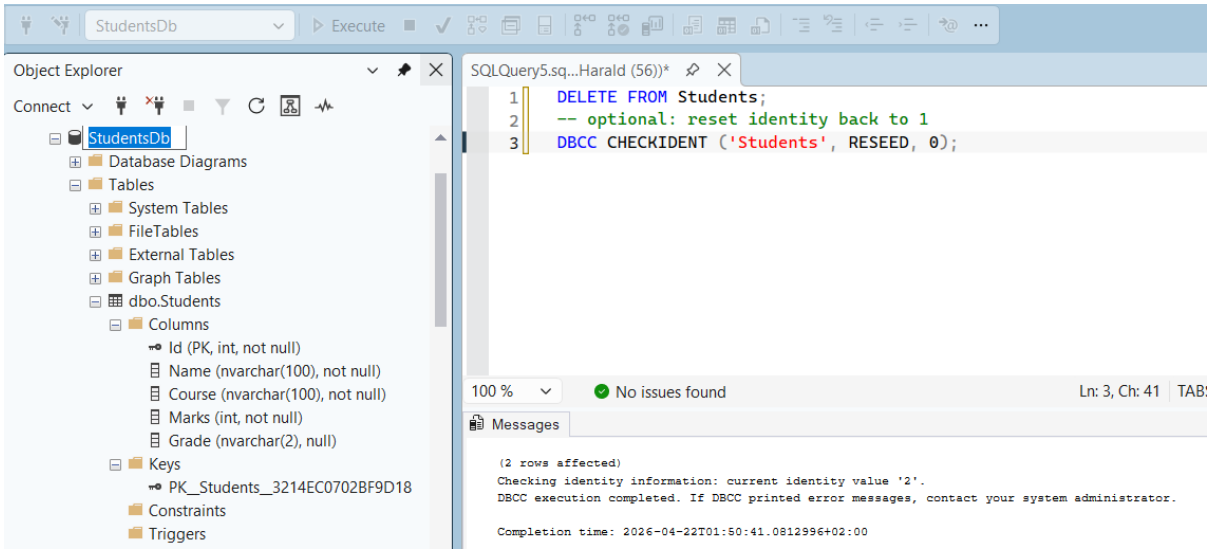

```
[
  {
    "id": 1,
    "name": "Alex Lee",
    "course": "DATA2410",
    "marks": 73,
    "grade": "C"
  },
  {
    "id": 3,
    "name": "Jane Smith",
    "course": "DATA2500",
    "marks": 92,
    "grade": "A"
  }
]
```

4.5 Clear the database (optional)

1. SSMS → connect to StudentsDb → New Query
2. Run:

   DELETE FROM Students;

   DBCC CHECKIDENT ('Students', RESEED, 0);
3. Verify in Scalar: GET /api/Students → should return [0]



5 Results

- The calculate-grades endpoint saves grades correctly based on the rules:
  - A ≥ 90, B = 80–89, C = 60–79, D < 60.
- The report endpoint shows the correct numbers per course:
  - totalStudents
  - averageMarks (2 decimals)
  - gradeDistribution (A/B/C/D)
- Example (from the latest test data):

```
  "courseName": "DATA2410",
  "totalStudents": 2,
  "averageMarks": 79,
  "gradeDistribution": {
    "a": 0,
    "b": 1,
    "c": 1,
    "d": 0
```

```
  "courseName": "DATA2500",
  "totalStudents": 1,
  "averageMarks": 92,
  "gradeDistribution": {
    "a": 1,
    "b": 0,
    "c": 0,
    "d": 0
```